CSWD QR Batch Generator

Feature Overview — City Social Welfare and Development Office · Generated June 22, 2026

1. Purpose

The CSWD QR Batch Generator is a web-based tool built for the City Social Welfare and Development Office to produce large sets of printable QR-code identification cards. Each card carries a unique six-digit control number that is encoded as the QR payload. Staff can request any quantity up to 10,000 cards from a single browser form; the system renders those cards into a US Letter PDF (or a ZIP of multiple PDFs for large batches) and initiates an immediate file download — no database write, no server-side storage of the generated file, and no page reload.

The primary use case is welfare-programme beneficiary registration, where each physical card is handed to one beneficiary and later scanned to confirm identity. Because the QR content is a plain control number, the cards carry no personally identifiable information at the point of printing; linking a card to a person happens downstream in a separate registry (see Section 8, Extension Points).

2. End-to-End Request Flow

2.1 Browser Form

The user visits the home page, which is served by `App\Controllers\Home` and rendered by `app/Views/home.php`. The form contains a single numeric input for the requested quantity and a submit button. Client-side validation (enforced by jQuery) prevents the form from submitting a value outside the range 1–10,000 before it even reaches the server.

2.2 AJAX Submission

Rather than a standard HTML form post, jQuery intercepts the submit event and sends the quantity to `POST /generate` as an XMLHttpRequest with `responseType` set to `'blob'`. This means the browser treats the response body as raw binary data, which is necessary because the server returns either a PDF or a ZIP file directly — there is no intermediate JSON envelope. When the response arrives, jQuery synthesises a temporary anchor element, sets its `href` to an object URL wrapping the blob, and programmatically clicks it to trigger the browser's native file-save dialog. The object URL is then revoked to release memory.

2.3 Controller Dispatch

The route resolves to `App\Controllers\Batch::generate()`. The controller reads the `quantity` field from the POST body, validates that it is an integer in the permitted range, and delegates all generation work to `App\Libraries\QrBatchPdfGenerator::generate()`. The controller is intentionally thin: it converts the library's return value into an HTTP response with the appropriate `Content-Type` header (`application/pdf` or `application/zip`), sets `Content-Disposition: attachment` with the correct filename, and returns.

2.4 QrBatchPdfGenerator — Chunked Rendering

`QrBatchPdfGenerator::generate()` first consults `QrBatchPlanner` to determine how many chunks the quantity requires. If the entire batch fits in one chunk (at most 50 pages, or 600 cards), it renders a single PDF and returns it directly. If the batch spans more than one chunk, it opens a temporary ZIP archive via PHP's built-in `ZipArchive` extension, renders each chunk sequentially, appends the resulting PDF bytes as a named entry (`batch-001.pdf`, `batch-002.pdf`, ...), and closes the archive. The temporary ZIP file lives in `sys_get_temp_dir()` and is unconditionally deleted in a `finally` block, even if a chunk render throws an exception mid-way.

Before rendering each chunk, the method raises the PHP memory limit to 512 MB if the current limit is lower. A full 50-page chunk with embedded SVG QR codes requires roughly 180 MB inside dompdf; the default web request limit of 128 MB would exhaust available memory before the render completes. The limit is raised only when necessary so as never to inadvertently shrink a higher pre-configured value. After each chunk, the local references to the chunk's PDF bytes are explicitly unset to release memory before the next chunk begins.

2.5 QR Image Generation

Individual QR codes are produced by `App\Libraries\QrImageGenerator::svgDataUri()`, which uses the `chillerlan/php-qrcode` library's `QRMarkupSVG` output driver. The driver encodes the SVG markup as a Base64 data URI suitable for embedding directly in an HTML `img` tag's `src` attribute. A fresh `QRCode` instance is created for every control number because reusing one instance across multiple `render()` calls causes the library to accumulate data segments internally, eventually exceeding the QR version's capacity.

The SVG format was chosen specifically because dompdf can render inline SVG without any native PHP image extensions. A PNG-based approach was originally planned but requires `ext-gd` to compose and encode the bitmap; the deployment environment does not have `ext-gd` available, so SVG was adopted instead. SVG QR codes also scale perfectly for print, producing sharp output at any physical size without resampling artifacts.

2.6 PDF Composition

Each page of a chunk is rendered from `app/Views/pdf/batch_page.php`, which receives an array of up to 12 cells. A cell is an associative array containing the formatted control number string and the SVG data URI. The view emits a fixed-layout HTML table that dompdf interprets as the print grid. Shared CSS for the grid is kept in `app/Views/pdf/_styles.php`, which is prepended once per chunk rather than once per page so that dompdf processes the style rules a single time. The final HTML string passed to dompdf is therefore the stylesheet followed by all page HTML concatenated in order.

3. Control-Number Scheme

Control numbers are sequential integers formatted as six-digit zero-padded decimal strings (for example, `000001`, `000042`, `001500`). The formatting is handled by `QrBatchPlanner::formatControlNumber()`, which uses PHP's `str_pad()` with a field width of 6 and a left-padding character of `'0'`.

For a given batch request the sequence always starts at 1 and increments by 1 for each card. Within a multi-chunk batch the `startNumber` argument passed to each chunk's render call advances by the number of cards already produced, so control numbers are globally unique and contiguous across all PDFs in the ZIP.

The string that is encoded into the QR code is exactly the formatted control number — plain ASCII text, no URL prefix, no metadata. Scanners therefore read back the bare number string, which downstream systems can match against a registry. This keeps the QR module count small (version 1 or 2) and maximises scan reliability.

4. Print and Layout Specifications

Property	Value	Notes
Paper size	US Letter (8.5 × 11 in)	Landscape not used; portrait only
Page margin	None (0)	`@page { margin: 0; }`
Grid layout	3 columns × 4 rows = 12 cells per page	Rendered as an HTML table; each column is 33.33% width, each row 25% page height
Cell borders	1 px dashed, color `#adb5bd`	Serve as cut guides; dashed pattern reduces toner usage
Cell padding	8 px top/bottom, 10 px left/right	
Header text	Purple (`#6f42c1`), bold, 10 px, centered	Programme name or office identifier printed at top of each cell
Field labels	8 px, dark (`#212529`), left-aligned	Name and other fields shown as label + blank underline for hand-writing
Blank underlines	70% cell width, `border-bottom: 1px solid`	Space for staff to write beneficiary details after printing
QR image size	1.1 × 1.1 in	Embedded as SVG data URI; vector, no raster scaling artifacts
Control-number label	8 px, dark	Static label "Control No." above the number
Control-number value	14 px, DejaVu Sans Mono (monospace)	Large, fixed-width for easy visual verification
Body font	Roboto (embedded TTF)	Registered per-render via dompdf's `FontMetrics::registerFont()`

5. Library Choices and Rationale

5.1 dompdf/dompdf

Dompdf converts an HTML+CSS string to a PDF byte stream entirely in pure PHP, with no external binary dependency. It was selected because the CI4 application is already PHP-only and the print layout is naturally expressible as a CSS table

grid. Dompdf's support for `@page` rules, table-based layout, and embedded fonts is sufficient for the fixed-grid card design. The library writes font metrics to a writable cache directory (`writable/fonts/`) to avoid repeating font-parsing work across requests.

5.2 chillerlan/php-qrcode

This library generates QR codes entirely in PHP without requiring any image or graphics extension. It supports multiple output formats through a driver interface; the `QRMarkupSVG` driver produces an inline XML SVG string that dompdf can embed directly. Error correction level M (roughly 15% data restoration capacity) is used — a reasonable balance between data density and scan tolerance for codes that will be printed and potentially handled physically. The library is well-maintained, follows semantic versioning, and is available through Composer.

5.3 ZipArchive (PHP built-in)

PHP's bundled `ZipArchive` extension handles multi-chunk batch assembly without any additional Composer dependency. Chunk PDFs are added to the archive as in-memory strings via `addFromString()`, which avoids writing individual chunk files to disk. Only the final assembled ZIP is written to a temporary file, read back as a byte string, and then deleted.

6. Scaling Notes

The system is designed to handle up to 10,000 cards per request. The key scaling constants, all defined in `QrBatchPlanner`, are:

- **12 cells per page** — a 3×4 grid on US Letter with zero margin.
- **50 pages per chunk** — each chunk is rendered into one PDF. 50 pages × 12 cards = 600 cards per PDF.
- **10,000 maximum quantity** — enforced at the controller. 10,000 cards require 834 pages, which is 17 chunks (PDFs in a ZIP).

Rendering is strictly sequential rather than parallel. PHP's single-threaded execution model and the shared memory space of a web worker process make parallel chunk rendering impractical without a queue or background-process architecture. For the expected workload (welfare programme batches of a few hundred to a few thousand), sequential rendering completes well within typical HTTP timeout limits.

The 512 MB memory ceiling is set per-render invocation and is released when the PHP process ends or is reused for the next request. If the deployment environment's `php.ini` already allows more than 512 MB, the code will never lower that limit.

7. Key Source Files

File	Role
`app/Controllers/Batch.php`	Validates quantity, calls generator, returns HTTP binary response

File	Role
`app/Libraries/QrBatchPdfGenerator.php`	Orchestrates chunked PDF rendering and ZIP assembly
`app/Libraries/QrBatchPlanner.php`	Pure-value class: constants, control-number formatting, chunk-count arithmetic
`app/Libraries/QrImageGenerator.php`	Wraps chillerlan to produce SVG data URIs per control number
`app/Views/pdf/_styles.php`	Shared CSS for the print grid; prepended once per chunk
`app/Views/pdf/batch_page.php`	Renders one page (up to 12 cells) of the QR grid
`app/Views/home.php`	Quantity form; jQuery AJAX wires up the blob download

8. Extension Points

8.1 Database-Backed Records

The current implementation is stateless: control numbers are generated on-the-fly and not persisted. A natural extension is to insert one row per generated control number into a beneficiary registry table at generation time, recording the number, batch ID, timestamp, and initially-null beneficiary fields. This would allow the office to track which numbers have been issued, detect duplicates, and pre-fill beneficiary data before printing.

8.2 URL-Encoded QR Payload

Instead of encoding a bare control number, the QR payload could be a URL pointing to a beneficiary lookup endpoint (for example, `https://cswd.example.gov.ph/verify/000042`). Scanning the code with any smartphone camera would then open a web page rather than displaying raw text. This requires no change to the PDF layout — only the string passed to `QrImageGenerator::svgDataUri()` changes. Note that URL payloads are longer and will push the QR to a higher version, making the module grid denser; error correction level M remains appropriate.

8.3 CSV Upload

For batches that must correspond to a pre-existing list of names or IDs, the form could accept a CSV file upload. The controller would parse the CSV rows, assign one control number per row, and pass the name or ID alongside the control number to the cell view so that the name field is pre-filled rather than left blank for hand-writing. The chunking and PDF rendering pipeline would be unchanged.